

```
# fuzzy_washer.py
# Fuzzy logic (Mamdani) for a washing machine:
# Inputs: Dirtiness, Load
# Output: Wash Time
# Membership functions are triangular per the assignment.
```

```
from dataclasses import dataclass
import numpy as np
```

```
@dataclass
```

```
class TriMF:
```

```
    a: float # left support
```

```
    b: float # peak
```

```
    c: float # right support
```

```
    def mu(self, x: float) -> float:
```

```
        if x <= self.a or x >= self.c:
```

```
            return 0.0
```

```
        if x == self.b:
```

```
            return 1.0
```

```
        if self.a < x < self.b:
```

```
            return (x - self.a) / (self.b - self.a)
```

```
        # self.b < x < self.c
```

```
        return (self.c - x) / (self.c - self.b)
```

```
# --- Define membership functions from the assignment ---
```

Dirtiness: Low (0–4, peak 2), Medium (2–6, peak 4), High (5–10, peak 7.5)

DIRT_LOW = TriMF(0, 2, 4)

DIRT_MED = TriMF(2, 4, 6)

DIRT_HIGH = TriMF(5, 7.5, 10)

Load: Small (0–5, peak 2.5), Large (4–10, peak 7)

LOAD_SMALL = TriMF(0, 2.5, 5)

LOAD_LARGE = TriMF(4, 7, 10)

Wash Time (minutes): Short (0–30, peak 15), Medium (20–50, peak 35), Long (40–90, peak 65)

TIME_SHORT = TriMF(0, 15, 30)

TIME_MEDIUM = TriMF(20, 35, 50)

TIME_LONG = TriMF(40, 65, 90)

def fuzzify_dirtiness(x):

return {

"Low": DIRT_LOW.mu(x),

"Med": DIRT_MED.mu(x),

"High": DIRT_HIGH.mu(x),

}

def fuzzify_load(x):

```

return {

    "Small": LOAD_SMALL.mu(x),

    "Large": LOAD_LARGE.mu(x),

}

```

```

def mamdani_inference(dirt, load):

```

```

    """

```

Rules (including R4 per assignment text):

R1: IF Dirtiness High AND Load Large -> Time Long

R2: IF Dirtiness Low AND Load Small -> Time Short

R3: IF Dirtiness Medium -> Time Medium

R4: IF Dirtiness High AND Load Small -> Time Medium

Returns firing strengths for each output label.

```

    """

```

```

    μ_high = dirt["High"]

```

```

    μ_med = dirt["Med"]

```

```

    μ_low = dirt["Low"]

```

```

    μ_small = load["Small"]

```

```

    μ_large = load["Large"]

```

```

    fire_R1 = min(μ_high, μ_large) # -> Long

```

```

    fire_R2 = min(μ_low, μ_small) # -> Short

```

```
fire_R3 =  $\mu$ _med          # -> Medium
```

```
fire_R4 = min( $\mu$ _high,  $\mu$ _small) # -> Medium
```

```
# Aggregate (max across rules per output label)
```

```
out = {
```

```
    "Short": fire_R2,
```

```
    "Medium": max(fire_R3, fire_R4),
```

```
    "Long": fire_R1
```

```
}
```

```
details = {"R1_Long": fire_R1, "R2_Short": fire_R2, "R3_Med": fire_R3, "R4_Med": fire_R4}
```

```
return out, details
```

```
def centroid_defuzzification(agg_levels, step=0.1):
```

```
    """
```

```
    Centroid of area for max-aggregated, clipped output MFs.
```

```
    agg_levels: dict with keys 'Short','Medium','Long' giving clip heights in [0,1].
```

```
    """
```

```
    y = np.arange(0, 90 + step, step)
```

```
    # For Max aggregation:  $\mu_{out}(y) = \max(\min(\mu_{label}(y), level\_label))$ 
```

```
    mu_short = np.minimum(np.array([TIME_SHORT.mu(yy) for yy in y]), agg_levels["Short"])
```

```
    mu_medium = np.minimum(np.array([TIME_MEDIUM.mu(yy) for yy in y]),  
agg_levels["Medium"])
```

```
    mu_long = np.minimum(np.array([TIME_LONG.mu(yy) for yy in y]), agg_levels["Long"])
```

```

mu_out = np.maximum.reduce([mu_short, mu_medium, mu_long])

numerator = np.trapz(y * mu_out, y)
denominator = np.trapz(mu_out, y)
crisp = numerator / denominator if denominator > 0 else float("nan")

return crisp, {"y": y, "mu_out": mu_out}

```

```
def main():
```

```
    # Inputs per assignment
```

```
    dirtiness = 6.0
```

```
    load = 3.0
```

```
    # (a) Fuzzification
```

```
    f_dirt = fuzzify_dirtiness(dirtiness)
```

```
    f_load = fuzzify_load(load)
```

```
    print("(a) Membership values")
```

```
    print(f" Dirtiness={dirtiness}: Low={f_dirt['Low']:.3f}, Med={f_dirt['Med']:.3f},  
High={f_dirt['High']:.3f}")
```

```
    print(f" Load={load}: Small={f_load['Small']:.3f}, Large={f_load['Large']:.3f}")
```

```
    # (b) Mamdani inference
```

```
    agg_levels, rule_fires = mamdani_inference(f_dirt, f_load)
```

```
    print("\n(b) Rule firing strengths")
```

```
for k, v in rule_fires.items():
```

```
    print(f" {k}: {v:.3f}")
```

```
print(" Aggregated output levels:", {k: round(v,3) for k,v in agg_levels.items()})
```

```
# (c) Defuzzification (Centroid)
```

```
crisp, _ = centroid_defuzzification(agg_levels, step=0.05)
```

```
print(f"\n(c) Defuzzified wash time (centroid): {crisp:.2f} minutes")
```

```
if __name__ == "__main__":
```

```
    main()
```